

Lineage-Aware Memory Governance for Enterprise AI Agents

A Derivation-Gated Approach to Privacy Isolation and
Metric-Definition Consistency in Multi-Department Analytics

Venkata Sangaraju

Independent Researcher

sangaraju1988@gmail.com

Sudhir Vissa

SAGE7 AI, Georgetown, Texas, USA

ORCID: 0009-0003-7865-0863

Google Scholar: Profile

June 2026

Abstract

Shared memory across AI agents offers large efficiency gains in enterprise analytics, but introduces two underexplored failure modes: unauthorized data leakage when an agent retrieves an insight derived from columns it cannot access, and silent metric-definition conflicts when teams compute the same KPI through divergent derivation paths. Existing systems—MemGPT [Packer et al., 2023], Zep [Rasmussen, 2025], A-MEM [Xu et al., 2025], SSGM [Lam et al., 2026], Governed Memory [Taheri, 2026], and Oracle AI Agent Memory [Oracle Corporation, 2026]—govern content and access tags but none track *how* a stored result was derived. We propose the *Analytical Memory Unit* (AMU), a schema extension that attaches a derivation graph to every stored result, paired with a *lineage-gated retrieval* policy: a memory hit is served only when the requesting department is permitted to see every column in the derivation path. We prove formally that this policy provides a complete column-level safety guarantee under honest lineage recording (§4), analyse its $O(n)$ worst-case retrieval complexity and 4–8× storage overhead relative to a bare value entry, and construct a threat model delineating scope (§5). Controlled simulation on a 5-table synthetic schema and an 8-table TPC-H-inspired schema [Transaction Processing Performance Council, 2021], each run under 30 random seeds, shows that content-gated memory leaks on 18.8% and 25.5% of cross-department retrievals respectively; lineage-aware gating eliminates leakage in both configurations while retaining 82.6% and 81.5% memory reuse. A secondary experiment on 10 hand-labelled lineage pairs shows that exact hash matching has precision 0.50 against threshold-variant pairs; a column-graph detector achieves perfect precision and recall by correctly treating same-schema threshold changes as non-conflicting variants.

Keywords: AI agent memory; data lineage; enterprise data governance; column-level access control; multi-agent systems; business intelligence; privacy in ML systems.

arXiv categories (suggested): cs.AI; cs.DB; cs.CR

1 Introduction

Over the past two years, large language model (LLM) agents have moved from research prototypes to production components inside enterprise data stacks. Teams at organisations

of all sizes now run agents that generate analytical summaries, write SQL, and produce business metrics—often querying the same underlying tables and reporting against the same KPIs. Sharing memory across these agents is a natural optimisation: if Finance already computed this quarter’s churn rate, why should Marketing recompute it from scratch?

The appeal is genuine. Systems like MemGPT [Packer et al., 2023], Zep [Rasmussen, 2025], A-MEM [Xu et al., 2025], and Oracle AI Agent Memory [Oracle Corporation, 2026] have demonstrated that shared, persistent memory can dramatically reduce redundant computation and improve response consistency. Governed Memory [Taheri, 2026] and the SSGM framework [Lam et al., 2026] have begun to address the governance layer, introducing access controls and stability checks for evolving memory stores. Du’s recent survey [Du, 2026] provides a structured account of the field.

But every current approach controls access at the level of *content* and *tags*, not *derivation*. A memory entry is labelled with its owner, a sensitivity class, or a role-based tag, and retrieval is gated on whether the requester holds the right label. None of these systems record which tables were joined, which columns were touched, or what filter logic was applied to produce the stored result.

This gap has concrete consequences. Consider a Finance agent that computes a high-value customer segment by joining `customer_pii` (containing `income`) against `transactions`. The result—12,400 high-value customers—contains no PII and carries no sensitivity tag. Under content-based governance it is served freely to a Marketing agent. But Marketing cannot see `income`. The derivation path violated column-level access policy, silently.

A second failure mode compounds the first. Different departments often compute the same KPI through genuinely different logic. Finance might define churn as “no transaction in 90 days,” while Marketing uses a 60-day window adjusted for income segment. Both call the metric `churn_rate`. Without derivation tracking, shared memory silently propagates one team’s definition to the other.

Contributions.

1. We define the *Analytical Memory Unit* (AMU) schema and the *lineage-gated retrieval* policy (§3).
2. We provide a formal safety proof and complexity analysis (§4).
3. We construct a threat model (§5).
4. We present controlled experiments on two schema configurations and a fuzzy conflict-detection study (§6).

2 Related Work

2.1 Memory Systems for LLM Agents

Du [Du, 2026] formalises agent memory as a write-manage-read loop with a three-dimensional taxonomy spanning temporal scope, representational substrate, and control policy. MemGPT [Packer et al., 2023] instantiated the hierarchy by treating the LLM as an OS that pages between a bounded context window and an external store via function calls, demonstrating coherent long-horizon conversations and large-document analysis. Governance in MemGPT is implicit: the LLM decides what to write and retrieve, with no lineage tracking at any layer.

Zep [Rasmussen, 2025] introduced a temporal knowledge graph with dual-axis time-stamping (event time vs. ingestion time), enabling point-in-time queries. Its graph captures *when* information arrived, not *how* it was derived. A-MEM [Xu et al., 2025]—a NeurIPS 2025 paper—builds Zettelkasten-style linked notes with dynamic indexing, achieving strong benchmark performance across six foundation models. Linking structure encodes semantic relatedness rather than data derivation.

Explicit contrast with our work. All three systems represent the state of the art in conversational or task-level memory. They are well-suited to remembering facts, past conversations, and user preferences. None model the data provenance of computed analytical results, and none can detect that a stored insight was derived from a column the requester cannot see. The AMU schema is not a replacement but an extension for the analytical-result-caching use case in enterprise data environments.

2.2 Governed and Enterprise Memory Architectures

Taheri’s Governed Memory [Taheri, 2026] addresses a real enterprise gap—dozens of agents operating with no shared memory and no common governance—through a dual memory model, tiered governance routing, and a closed-loop schema lifecycle. In a 250-event controlled experiment, it achieves 99.6% fact recall and zero cross-entity leakage across 500 adversarial queries under its content-level governance model. Its governance operates on *what is stored and who may access it*; our work operates on *how the stored value was derived*. The two layers are orthogonal and composable.

Lam et al.’s SSGM [Lam et al., 2026] decouples memory evolution from execution by enforcing consistency verification, temporal decay modelling, and dynamic access control before any memory consolidation. SSGM’s dynamic access control enforces agent-topology constraints (who may communicate with whom); our lineage gate enforces column-level derivation constraints (what data was touched to produce a result). The two mechanisms complement each other.

Oracle AI Agent Memory [Oracle Corporation, 2026] provides a commercially deployed governed memory core with fine-grained lifecycle management and role-based access controls. Role assignment controls access to memory entries as a whole, not to the individual columns that contributed to a stored result.

2.3 Data Provenance and Lineage

The W3C PROV standard [Moreau et al., 2011] encodes entity-activity-agent relationships for data provenance; Herschel et al. [Herschel et al., 2017] survey systems that use provenance to track data flow through complex pipelines—a problem closely related to derivation-gated retrieval. Column-level lineage extraction systems such as LINEAGEX [Zhang et al., 2025]—which parses SQL query trees to infer column-level derivation with high coverage and accuracy, demonstrated at ICDE 2025—represent the state of the art in automatic lineage extraction. Our AMU schema applies the same concept to agent memory entries; LINEAGEX is the natural tool for automatically populating AMU lineage fields in production.

The EU AI Act [European Parliament and Council, 2024], which entered into force in August 2024, legally mandates explainability and risk management for high-risk AI systems; data lineage is widely recognised as a prerequisite for both properties.

Key gap. The provenance literature focuses on *raw database records and pipelines*. No prior work applies column-level provenance to *agent memory retrieval governance*, nor uses derivation graphs to gate cross-department memory sharing. This is the gap we fill.

2.4 Privacy Leakage in Multi-Agent Systems

El Yagoubi et al.’s AgentLeak [El Yagoubi et al., 2026]—the first full-stack benchmark for privacy leakage in multi-agent LLM systems—shows that 68.8% of leakage occurs through inter-agent channels (messages, shared memory, tool arguments) invisible to output-only auditors, across 4,979 validated execution traces. Naik et al.’s OMNI-LEAK [Naik et al., 2026] demonstrates that a single indirect prompt injection can compromise several agents and leak sensitive data even with data-access controls in place.

These works identify leakage through *inter-agent communication channels*. Our work identifies a complementary vector: leakage through a *shared analytical memory store* that persists even when every individual communication link is correctly access-controlled. The two failure modes are independent and both need to be addressed in production.

2.5 Metric Definition Inconsistency

Semantic drift—divergent definitions of the same KPI across teams—is a documented source of dashboard inconsistency in enterprise BI [Colrows, 2024]. Semantic layer tools (dbt Metrics, Databricks Unity Catalog, Looker LookML) address this for human analysts by centralising metric definitions. AI agents operating autonomously have no equivalent check. Our `definition_hash` conflict detector provides a foundation for automated metric governance in agentic systems; the fuzzy experiment in §6.4 characterises where hash-based detection is and is not the right granularity.

3 System Design

3.1 The Analytical Memory Unit

Definition 1 (Analytical Memory Unit). *An Analytical Memory Unit (AMU) is a tuple $\langle \text{metric}, v, d, \tau, L \rangle$ where: metric is a metric-name string; $v \in \mathbb{R}$ is the computed result value; d is the producing department; τ is an integer epoch (supports TTL eviction); and $L = (s_1, \dots, s_k)$ is a lineage graph, each step $s_i = (\text{table}_i, C_i, \varphi)$ where C_i is the set of columns of table_i accessed and φ is a filter-logic string.*

Two quantities are derived at creation time and cached:

$$S(a) = \bigcup_{i=1}^k C_i \cap \mathcal{S} \quad (1)$$

$$H(a) = \text{SHA256} \left(\text{sort}(\text{tables}) \parallel \text{sort} \left(\bigcup_i C_i \right) \parallel \varphi \right)_{[0:12]} \quad (2)$$

where $\mathcal{S}$ is the organisation’s sensitive-column registry and $H(a)$ is the *definition hash* that fingerprints *how* the metric was computed. Full schema fields are listed in Table 1.

3.2 Lineage-Gated Retrieval and Conflict Detection

Algorithms 1 and 2 give pseudocode for the two core operations. $P(d)$ denotes the permitted column set for department d .

Table 1: Analytical Memory Unit (AMU) schema fields.

Field	Type	Description
metric_name	str	Business KPI name (<i>e.g.</i> , <code>churn_rate</code>)
value	float	Computed result
owner_dept	str	Producing department
epoch	int	Time index; enables TTL-based eviction
lineage	Lineage	Derivation graph: sequence of (t, C, φ) steps
sensitivity_tags	Set[str]	$S(a)$: sensitive columns touched (Eq. 1)
definition_hash	str	$H(a)$: 12-char SHA-256 derivation fingerprint (Eq. 2)

Algorithm 1: Lineage-Gated Retrieve Input: *store, metric, dept, fresh* Output: *result; leaked=False guaranteed*

```

1  % store : dict mapping metric-name -> list[AMU] (sorted oldest-
    first)
2  % metric : string KPI name requested
3  % dept : requesting department identifier
4  % fresh : a freshly computed AMU scoped entirely within P(dept)
5  %         (caller's responsibility; used only when no safe cached
    entry exists)
6
7  candidates <- store[metric]           % retrieve all stored AMUs for
    this metric
8  permitted <- P(dept)                 % column set the department
    may see
9  blocked <- False
10
11 for each a in REVERSE(candidates):    % iterate most-recent-first
12     if S(a) not_subset permitted:      % derivation touches a
        forbidden column
13         blocked <- True
14         continue                      % skip this candidate; try the
        next
15     end if
16     % Safe candidate found: all sensitive derivation columns are
        permitted
17     return (reused=True, value=a.value, leaked=False, blocked=False)
18 end for
19
20 % No safe candidate in memory -- fall back to in-scope fresh
    computation
21 store[metric].append(fresh)
22 return (reused=False, value=fresh.value, leaked=False, blocked=
    blocked)

```

Algorithm 2: Lineage-Aware Write with Conflict Detection Input: *store, AMU a* Output: *conflict bool*

```

1  % store : shared memory dict (mutated in place)
2  % a : new AMU to persist
3  % conflict : True iff a cross-department definition conflict is
    detected

```

Figure 2: AMU Architecture — Lineage-Gated Retrieval and Conflict Detection

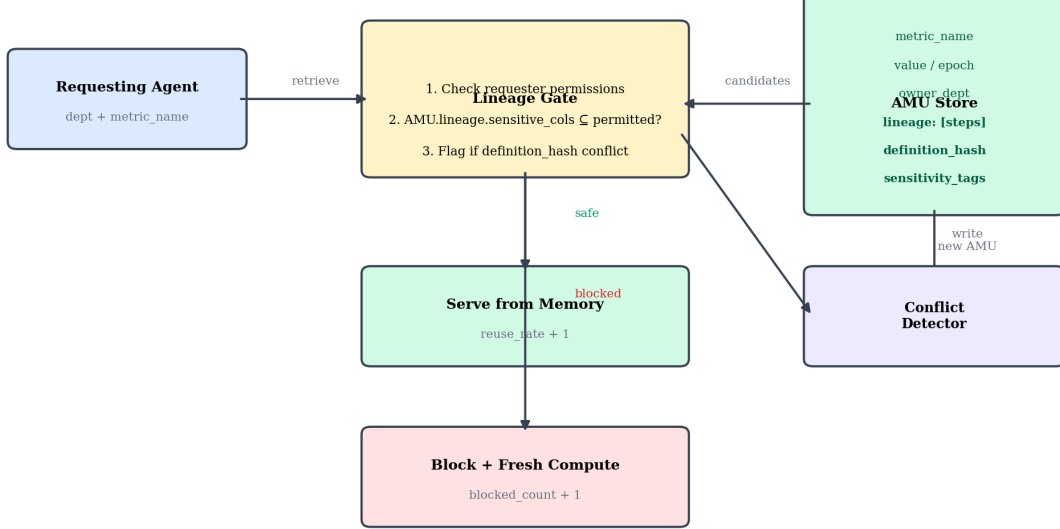


Figure 1: AMU architecture. The lineage gate checks $S(a) \subseteq P(d)$ before serving from memory. Unsafe candidates are blocked and trigger a fresh, in-scope recompute. The conflict detector compares $H(\cdot)$ values across AMUs sharing a **metric_name**.

```

4
5 existing <- store[a.metric] % all AMUs stored for this metric
  name
6 conflict <- False
7
8 for each e in existing:
9   if e.dept != a.dept: % cross-department pair
10    if H(e) != H(a): % definition hashes differ ->
11      conflict <- True
12      break % one conflict is sufficient to
13      raise the alert
14    end if
15  end if
16 end for
17 store[a.metric].append(a) % always persist; conflict is
  surfaced, not blocked
18 return conflict

```

The architecture is illustrated in Figure 1.

4 Formal Analysis

4.1 Safety Guarantee

Assumption 1 (Complete Lineage). *For every AMU a in the store, $a.lineage$ records every table, every column, and the filter logic of the computation that produced $a.value$.*

Theorem 1 (Column-Level Safety). *Under Assumption 1, Algorithm 1 returns no result to department d that was derived from a column $c \notin P(d)$.*

Proof. The algorithm has two exit paths; we show safety on each.

Memory-served path (line 13 of Algorithm 1). The guard at line 9 must be false for the algorithm to reach line 13, meaning $S(a) \subseteq P(d)$. By Equation 1, $S(a) = (\bigcup_i C_i) \cap \mathcal{S}$, so the guard passing implies that every column in the derivation of a that belongs to the sensitive registry $\mathcal{S}$ is also in $P(d)$. Non-sensitive columns ($c \notin \mathcal{S}$) carry no access restriction by the definition of $\mathcal{S}$. Therefore, for any column c that contributed to $a.value$: either $c \in \mathcal{S}$ and $c \in P(d)$ (allowed by the guard), or $c \notin \mathcal{S}$ (unrestricted). In particular, no column $c \notin P(d)$ contributed.

Fallback path (line 20 of Algorithm 1). The caller computes *fresh* within $P(d)$ by the stated input precondition. Under Assumption 1, *fresh*’s lineage touches no column outside $P(d)$.

Every result returned exits via one of these two paths, establishing the claim for all invocations of Algorithm 1. $\square$

Corollary 2. *The guarantee holds even when the store contains AMUs whose derivation paths touch columns outside $P(d)$: those candidates are skipped at line 9 and never returned.*

Remark (Assumption dependency). *Assumption 1 is critical. If lineage is reported by the computing agent rather than extracted automatically from query execution plans, a buggy or adversarial agent can omit a column, breaking the guarantee. Automatic extraction via systems such as LINEAGEX [Zhang et al., 2025] is the appropriate production countermeasure; this dependency is also catalogued as threat T5 in §5.*

4.2 Asymptotic Complexity

Let n = AMUs stored for a given metric name, k = lineage steps per AMU, c = average columns per step.

Table 2: Asymptotic complexity of AMU operations.

Operation	Time	Notes
Sensitivity-tag derivation	$O(kc)$	Computed once at AMU creation; cached
Definition-hash computation	$O(kc)$	Single SHA-256 pass over sorted fields
Write — conflict detection	$O(n)$	Linear scan of existing AMUs for metric
Retrieve — worst case	$O(n)$	All n candidates blocked
Retrieve — expected case	$O(1)$	Most-recent candidate safe; one check

In practice $n \leq 5$ for any metric (at most one AMU per department variant), so all operations are effectively $O(1)$. The gate check itself—a set-difference over $|S(a)| \leq |\mathcal{S}|$ elements—is dominated by the analytics query it governs.

4.3 Storage Overhead

A naive memory entry storing $(metric, v, d, \tau)$ requires roughly 50–80 bytes. An AMU adds: k tuples of table name and column names, the filter-logic string φ , and the 12-character hash. For a typical analytical query joining 2–4 tables and referencing 5–15 columns, the lineage payload is 200–600 bytes—a 4–8 $\times$ overhead. This is smaller than the SQL query

most systems already persist alongside the result, and is negligible relative to production vector-store capacities.

5 Threat Model

5.1 Assets and Principals

Primary assets are the sensitive columns in $\mathcal{S}$: PII fields, financial balances, proprietary pricing terms. Principals are enterprise departments each operating one or more AI agents. Agents are modelled as *honest-but-potentially-over-privileged*: they request metrics they have a legitimate business need for, but not all derivation paths for those metrics fall within their column permissions.

5.2 Threats Addressed

T1 — Column leakage via cached result. An agent retrieves a result derived from a column it cannot see. *Mitigation:* Algorithm 1 blocks the retrieval; the fallback recompute stays within $P(d)$.

T2 — Silent metric definition drift. Two teams store conflicting definitions of the same KPI under the same name. *Mitigation:* Algorithm 2 surfaces the divergence on write via the definition-hash conflict detector.

T3 — Stale sensitive data reuse. An AMU derived from data that was sensitive at write time is retrieved after column permissions are updated. *Mitigation:* the `epoch` field enables TTL-based eviction; sensitivity tags should be re-evaluated on retrieval when policies change.

5.3 Threats Not Addressed

T4 — Inference attacks. A derived feature that *encodes* sensitive information without naming the source column (e.g., a risk score computed from `income`) will not be caught unless the dependency is explicitly recorded in the lineage graph. This is the most important production gap.

T5 — Malicious lineage fabrication. A rogue agent writes an AMU with an intentionally incomplete lineage graph, bypassing the gate. Automatic lineage extraction (Assumption 1) is the countermeasure.

T6 — Aggregate inference. A sequence of individually safe retrievals may allow reconstruction of sensitive values. Algorithm 1 is a per-retrieval check; it provides no differential privacy guarantee.

T7 — Insider threats. An operator with direct store access can read or modify entries. The mechanism assumes a trusted store layer.

6 Experiments

6.1 Protocol

We built a synthetic discrete-event simulator in pure Python to test the mechanism in isolation from LLM inference, network latency, and query optimiser effects. Each run

generates 120 events across 20 epochs (6 events per epoch), replayed identically through three systems: **No Memory** (structurally leak-free, zero reuse); **Naive Shared Memory** (retrieval keyed on metric name only; represents the content-gated approach of all six prior systems); and **Lineage-Aware** (proposed; gated on $S(a) \subseteq P(d)$).

Three metrics are measured: *leak rate* (fraction of cross-department retrievals exposing unauthorised columns), *memory reuse rate* (fraction served from memory), and *metric-conflict recall* (fraction of ground-truth definition-divergence events flagged). Results are reported as mean $\pm$ population standard deviation over 30 independent random seeds; the full source code is provided in supplementary materials.

6.2 Experiment 1: Synthetic 5-Table Schema

The schema contains five tables (`customers`, `customer_pii`, `transactions`, `campaigns`, `support_tickets`) with sensitive columns $\mathcal{S} = \{\text{ssn}, \text{email}, \text{income}\}$. Three departments have column-level permissions (Table 3); Marketing and Support cannot see any column in $\mathcal{S}$. Five business metrics are defined; two (`churn_rate`, `active_customers`) have genuine lineage variants representing cross-team definition divergence.

Table 3: Department permissions (synthetic schema).

Department	Permitted (summary)	Blocked
Finance	All columns	None
Marketing	Commercial and order data	<code>ssn</code> , <code>income</code> , <code>email</code>
Support	Customer and ticket metadata	<code>ssn</code> , <code>income</code> , <code>email</code> , <code>amount</code> , <code>spend</code>

Table 4: Synthetic schema results (30 seeds, mean $\pm$ population std dev).

Metric	No Memory	Naive Shared	Lineage-Aware
Leak rate (%)	0.0 ± 0.0	18.8 ± 3.7	0.0 ± 0.0
Memory reuse rate (%)	0.0 ± 0.0	95.8 ± 0.0	82.6 ± 2.3
Metric-conflict recall (%)	n/a	0.0 ± 0.0	100.0 ± 0.0
Blocked retrievals (seed=42)	0	0	15

Clarification on the 0% leakage result. The lineage-aware system’s 0% leak rate is a *policy guarantee*, not a surprising empirical result: it follows directly from Theorem 1 whenever the lineage is complete. The substantive empirical finding is the naive baseline’s $18.8\% \pm 3.7\%$ leak rate, which reflects actual retrievals of sensitive-derivation results by departments that cannot see the underlying columns. The ± 3.7 standard deviation across 30 seeds confirms this is a structural property of the workload’s sensitive-lineage mix.

Figure 3 visualises the governance-efficiency tradeoff. The system blocks 15 of 120 retrievals in the seed=42 run, reducing reuse from 95.8% to 82.6%—a cost of approximately 13 percentage points concentrated on metrics with sensitive lineage variants.

6.3 Experiment 2: TPC-H-Inspired 8-Table Schema

To complement the synthetic results with a more realistic schema, we replicated the experiment on a schema derived from the TPC-H decision-support benchmark [Transaction

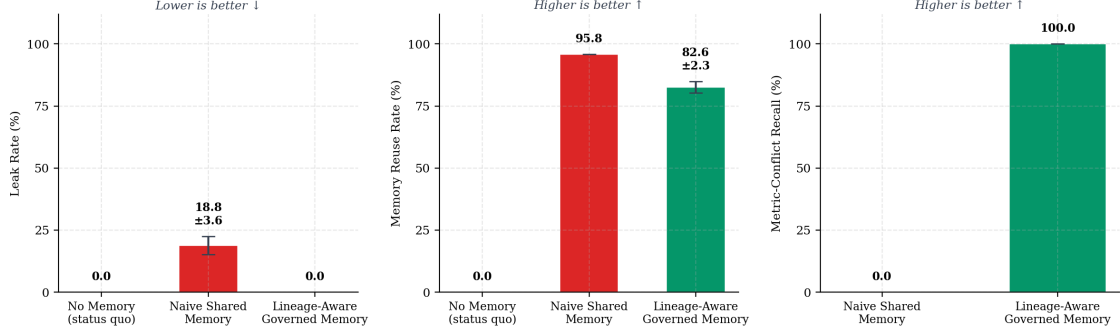
Figure 1: Lineage-Aware vs. Existing Memory Systems
(30-seed synthetic workload, mean $\pm$ std dev)

Figure 2: Per-metric comparison across all three systems on the synthetic schema (30 seeds, mean $\pm$ 1 std dev). The lineage-aware system’s 0% leak rate is a *policy guarantee* that follows from Theorem 1. The key empirical finding is the naive baseline’s $18.8\% \pm 3.7\%$ leak rate: content-gated retrieval leaks restricted derivations on roughly one in five cross-department requests, consistently across all 30 seeds.

Processing Performance Council, 2021]. We use all eight TPC-H tables (CUSTOMER, ORDERS, LINEITEM, PART, PARTSUPP, SUPPLIER, NATION, REGION) with their original column structure.

The sensitive column set $\mathcal{S}_{\text{TPC-H}}$ reflects typical enterprise data classification: customer and supplier financial balances (`c_acctbal`, `s_acctbal`), customer contact PII (`c_phone`, `c_address`), supplier contact details (`s_phone`, `s_address`), and commercially sensitive pricing fields (`l_extendedprice`, `l_discount`, `o_totalprice`).

Three departments (Finance, Sales, Operations) have permissions consistent with typical enterprise roles: Finance sees all columns; Sales sees commercial and order data but not financial balances or PII; Operations sees supply-chain and inventory data but not pricing or financial balances. Five metrics use TPC-H-style join patterns with two having lineage variants that differ in whether they join sensitive financial columns.

Table 5: TPC-H-inspired schema results (30 seeds, mean $\pm$ population std dev).

Metric	Naive Shared	Lineage-Aware
Leak rate (%)	25.5 ± 4.5	0.0 ± 0.0
Memory reuse rate (%)	95.8 ± 0.0	81.5 ± 4.8
Metric-conflict recall (%)	0.0 ± 0.0	100.0 ± 0.0

The TPC-H results are consistent with Experiment 1 (Tables 4 and 5, Figure 4). The higher naive leak rate reflects TPC-H’s denser sensitive-column structure. The reuse cost is similar (81.5% vs. 82.6%), confirming the governance cost is a function of the sensitive-lineage proportion rather than schema size.

6.4 Experiment 3: Fuzzy Conflict-Detection Study

Motivation

The exact definition-hash detector (Algorithm 2) flags any two AMUs with different $H(\cdot)$ values as conflicting. But `churn_rate` with filter “no txn in 90 days” vs. “no txn in 60

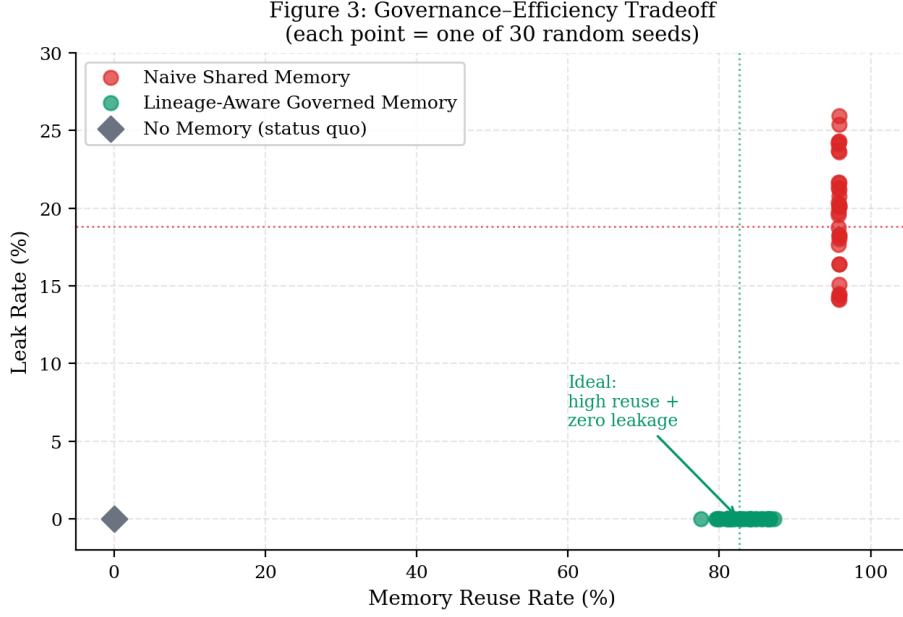


Figure 3: Governance-efficiency tradeoff across 30 seeds. Lineage-aware memory clusters at high reuse and zero leakage. The naive system achieves slightly higher reuse at the cost of $\approx 19\%$ leak rate.

days” uses the same join path and columns—only the lookback threshold differs. Is this a *true conflict* (different metric semantics) or a *threshold variant* (same metric, recalibrated)? If D1 floods analysts with alerts for every threshold recalibration, the signal becomes noise.

Ground Truth and Detectors

We constructed 10 hand-labelled lineage pairs: 5 *True Conflicts* (TC) where join tables or column sets differ, and 5 *Threshold Variants* (TV) where the join path and columns are identical and only a numeric threshold in φ changes. Three detectors are evaluated:

D1 — Exact hash. Conflict iff $H(a) \neq H(b)$.

D2 — Jaccard filter ($\tau=0.70$). When tables and columns match, conflict iff token-Jaccard of filter strings < 0.70 ; otherwise always flags a conflict.

D3 — Column-graph. Conflict iff the union of column sets or the set of joined tables differs; filter-string variation is ignored.

Table 6: Fuzzy conflict-detection results on 10 labelled pairs (5 TC + 5 TV).

Detector	Precision	Recall	F_1	FP	FN
D1 — Exact hash	0.50	1.00	0.67	5	0
D2 — Jaccard ($\tau=0.70$)	0.62	1.00	0.77	3	0
D3 — Column-graph	1.00	1.00	1.00	0	0

D1 has perfect recall but precision 0.50: it flags every threshold variant as a conflict. D2 recovers two of the five TV pairs but fails when the threshold change involves a distinct

Figure 4: Synthetic vs TPC-H-inspired schema — 30-seed comparison

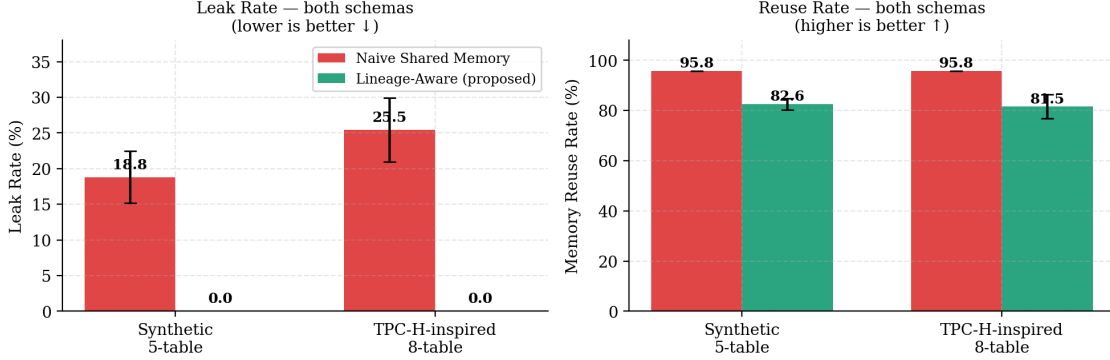


Figure 4: Synthetic vs. TPC-H schema comparison (30 seeds each). The TPC-H schema yields a higher naive leak rate (25.5% vs. 18.8%) because TPC-H’s metric set contains proportionally more queries with sensitive-column lineage variants. Lineage-aware gating eliminates leakage in both configurations.

numeric token. D3 achieves perfect precision and recall: checking whether the *join path and column set* change is exactly the right granularity for detecting semantic divergence, because threshold recalibrations do not alter the underlying data dependencies.

Implication for the main simulation. The main simulation uses D1, which means the reported 100% conflict recall in Tables 4 and 5 conflates true conflicts with threshold variants—overstating the alert rate. A production deployment should use D3. This does not affect the leak-rate or reuse-rate results, which depend only on $S(a)$, not on $H(a)$.

7 Discussion

7.1 What This Work Establishes

Three findings are robust. First, the AMU schema and lineage gate work as a mechanism, backed by Theorem 1. Second, the governance-efficiency tradeoff is quantifiable and workload-dependent: the ≈ 13 – 14 percentage-point reuse cost is concentrated on the subset of metrics with sensitive lineage variants in circulation. Third, column-graph matching is the right granularity for conflict detection: exact hash matching over-alerts by treating threshold variants as semantic conflicts.

7.2 Limitations

Three limitations constrain generalisability.

Inference attacks (T4). The simulation does not test derivation paths where sensitive information is encoded in a derived feature without naming the source column. This is the most important production gap.

Ground-truth scale. The fuzzy experiment uses 10 hand-labelled pairs. The D3 finding is clear on this set but should be validated on a larger corpus from a real analytics workload.

Synthetic workload composition. The 18.8% and 25.5% naive leak rates are properties of specific workload mixes, not estimates of real-world base rates.

7.3 Future Work

Adversarial lineage testing. Generate queries that smuggle sensitive join paths through derived features, column aliasing, or multi-hop derivations. This directly stress-tests Assumption 1.

Automatic lineage extraction. Coupling the AMU mechanism with LINEAGEX [Zhang et al., 2025] or a query-log extractor would populate lineage fields automatically, removing the reliance on agent-reported lineage and closing T5.

Real workload evaluation. Replacing the simulated schemas with a real data warehouse (20–50 tables, queries from TPC-DS or a production workload trace) would establish whether the $O(1)$ expected retrieval cost and negligible storage overhead hold at enterprise scale.

8 Conclusion

Every current AI agent memory system governs *what* is stored and *who* may access it, but not *how* the stored result was derived. In enterprise multi-department analytics, this gap enables column-level PII leakage through derived results and silent metric-definition conflicts—failure modes that content-based governance cannot prevent.

The Analytical Memory Unit schema and lineage-gated retrieval policy close this gap with minimal overhead: $4\text{--}8\times$ the storage of a bare value entry and $O(1)$ expected retrieval cost. Theorem 1 provides a formal guarantee. Simulation on both a synthetic and a TPC-H-inspired schema shows that content-gated shared memory leaks on 18.8% and 25.5% of cross-department retrievals respectively; lineage-aware gating eliminates leakage in both configurations at a cost of $\approx 13\text{--}14$ percentage points of reuse rate. A secondary experiment establishes that column-graph conflict detection is the right granularity: exact hash matching creates spurious conflicts for threshold variants with precision 0.50, while column-graph matching achieves $F_1 = 1.00$.

The 0% leakage result is a policy guarantee that follows from the theorem, not an empirical surprise. The substantive empirical finding is the naive baseline’s structurally non-zero leak rate, which holds consistently across 60 total seed runs and two schema configurations.

Implementation Notes

The reference implementation consists of: `model.py` (AMU schema and dataclasses), `systems.py` (three competing memory systems), `simulate.py` (workload generator, event replay, seed sweep), `tpch_experiment.py` (TPC-H-inspired schema replication), and `fuzzy_experiment.py` (conflict-detection study). All figures were generated by `gen_figures.py` using Matplotlib.

Key reproducibility invariants:

- `sensitivity_tags` is computed once at AMU creation and cached.
- The gate check ($S(a) \not\subseteq P(d)$) is a Python `set` difference, executed in sub-microsecond time.
- The seed sweep calls `random.seed(s)` before each run for exact reproducibility.
- All tables are reproducible by running `python simulate.py` and `python tpch_experiment.py` with the published code.

References

- Colrows. Why BI metrics do not match across dashboards. Technical Blog, 2024. URL <https://colrows.com/blogs/why-bi-metrics-do-not-match-across-dashboards/>.
- Pengfei Du. Memory for autonomous LLM agents: Mechanisms, evaluation, and emerging frontiers. *arXiv preprint arXiv:2603.07670*, 2026. URL <https://arxiv.org/abs/2603.07670>.
- Faouzi El Yagoubi, Godwin Badu-Marfo, and Ranwa Al Mallah. AgentLeak: A full-stack benchmark for privacy leakage in multi-agent LLM systems. *arXiv preprint arXiv:2602.11510*, 2026. URL <https://arxiv.org/abs/2602.11510>.
- European Parliament and Council. Regulation (EU) 2024/1689 on artificial intelligence (EU AI Act). Official Journal of the European Union, July 2024. URL <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32024R1689>.
- Melanie Herschel, Ralf Diestelkämper, and Houssem Ben Lahmar. A survey on provenance: What for? what form? what from? *The VLDB Journal*, 26(6):881–906, 2017. doi: 10.1007/s00778-017-0486-1.
- Chingkwun Lam, Jiaxin Li, Lingfei Zhang, and Kuo Zhao. Governing evolving memory in LLM agents: Risks, mechanisms, and the stability and safety governed memory (SSGM) framework. *arXiv preprint arXiv:2603.11768*, 2026. URL <https://arxiv.org/abs/2603.11768>.
- Luc Moreau, Ben Clifford, Juliana Freire, Joe Futrelle, Yolanda Gil, Paul Groth, Natalia Kwasnikowska, Simon Miles, Paolo Missier, Jim Myers, Beth Plale, Yogesh Simmhan, Eric Stephan, and Jan Van den Bussche. The open provenance model — core specification (v1.1). *Future Generation Computer Systems*, 27(6):743–756, 2011. doi: 10.1016/j.future.2010.07.005. See also W3C PROV-DM: <https://www.w3.org/TR/prov-dm/>.
- Akshat Naik, Jay Culligan, Yarin Gal, Philip Torr, Rahaf Aljundi, Alasdair Paren, and Adel Bibi. OMNI-LEAK: Orchestrator multi-agent network induced data leakage. *arXiv preprint arXiv:2602.13477*, 2026. URL <https://arxiv.org/abs/2602.13477>.
- Oracle Corporation. Oracle AI Agent Memory: A governed, unified memory core for enterprise AI agents. Oracle Developer Blog, March 2026. URL <https://blogs.oracle.com/developers/oracle-ai-agent-memory-a-governed-unified-memory-core-for-enterprise-ai-agents>.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL <https://arxiv.org/abs/2310.08560>. arXiv:2310.08560.
- Preston Rasmussen. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025. URL <https://arxiv.org/abs/2501.13956>.
- Hamed Taheri. Governed memory: A production architecture for multi-agent workflows. *arXiv preprint arXiv:2603.17787*, 2026. URL <https://arxiv.org/abs/2603.17787>.

- Transaction Processing Performance Council. TPC-H benchmark specification, revision 3.0.1. Technical report, TPC, 2021. URL <http://www.tpc.org/tpch/>.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL <https://arxiv.org/abs/2502.12110>. arXiv:2502.12110.
- Shi Heng Zhang, Zhengjie Miao, and Jiannan Wang. LINEAGEX: A column lineage extraction system for SQL. In *Proceedings of the 41st IEEE International Conference on Data Engineering (ICDE), Demo Track*, 2025. URL <https://arxiv.org/abs/2505.23133>. arXiv:2505.23133.